

ECHORA

Real Implementation Analysis and Test Evidence

This report is evidence-based: statements about behaviour are traced to the current codebase, while test status records the actual outcome of the test command executed in this workspace.

1. Scope and evidence

The analysis reviewed the live source code under `src/`, the active configuration, the runtime tests under `tests/`, and the result of running `pytest`. It does not claim that camera hardware, ML inference, Ollama, an ESP32 wristband, or audio hardware were validated because those integrations were not runnable in the available test environment.

Evidence type	What it establishes	Status
Source inspection	Current data flow, thresholds, module dependencies, and integration paths	Completed
Test inventory	23 <code>pytest</code> runtime test functions across OCR, obstacles, interaction, banknotes, and audio	Completed
Automated test execution	<code>pytest</code> collection attempted using bundled Python	Blocked by missing dependencies
Hardware/model validation	OAK-D, trained models, TTS, VLM, ESP32	Not executed

Architecture in one sentence

ECHORA is a Python real-time control loop that turns synchronized RGB, depth, and motion data into feature-specific AI results, then selects the safest feedback mode and communicates through speech, spatial sound, and haptics.

2. Real runtime architecture

The application starts in `main.py` and constructs `ControlUnit`. Startup initializes the OAK-D pipeline, PaddleOCR, dual YOLO obstacle detection, MediaPipe Hands, a banknote model, SQLite, InsightFace, haptic services, audio, and the `StateMachine`. The `ControlUnit` then loops over synchronized camera bundles.

Actual per-frame ordering

- Read RGB frame, aligned depth map, IMU sample, and timestamp from the OAK-D pipeline.
- Run the obstacle detector first and obtain tracked obstacle and interaction-object detections.
- Run the state machine using probe results retained from previous sampling ticks.
- Launch or update periodic probes for OCR, face presence, banknote presence, and interactable-object distance.
- Execute the active mode handler: Navigation, OCR, Interaction, Face ID, or Banknote.
- Draw a debug overlay, collect frame timing, and process user keys when a display is enabled.

Mode selection is safety-first

Navigation is the default. OCR requires text within 2 m and low motion; Interaction requires an interactable object within 0.8 m; Face ID requires repeated confident face detection; Banknote requires repeated note detection and low motion. The `StateMachine` uses consecutive-frame counters and dwell times. A danger obstacle forces a return to Navigation in all specialized modes except the person-only Face ID exception.

Observed implementation detail

The state machine is evaluated before the current iteration updates its probes. Therefore a new OCR, face, banknote, or interaction signal is normally consumed on a later frame. This is a small, deliberate-looking latency trade-off that keeps heavyweight probes asynchronous, but it should be described as sampled rather than instantaneous behaviour.

3. Feature-level analysis

Feature	Actual implementation	Feedback/output	Test coverage
Navigation	Custom accessibility YOLOv8 plus optional COCO YOLOv8; depth sampled per bounding box; ByteTracker confirms tracks and estimates movement.	Closest danger/warning object is spoken with direction; danger adds spatial sound and checkerboard haptic pulse.	7 runtime tests cover corridor promotion, unknown depth, dual-model merge, track confirmation/expiry, and VLM failure disabling.
OCR	PaddleOCR PP-OCRv4 with optional Arabic and English models. Uses contrast correction, skew correction, sharpening, denoising, caching, ordering, and stability history.	Stable text is spoken. After about 5 s with no text, a repositioning prompt is spoken.	9 runtime tests cover language choice, GPU fallback, cache behaviour, normalization, distance priority, and document order.
Face ID	InsightFace buffalo_sc: RetinaFace detection and ArcFace 512-D embeddings. Cosine similarity plus runner-up margin; SQLite stores profiles and events.	Known person or unknown-person speech; successful identification updates seen count and event log.	No active pytest runtime test file. Legacy self-test is excluded by conftest.
Banknote	Dedicated custom YOLOv8 model. Highest-confidence label enters a small temporal history before confirmation.	Stable Egyptian Pound denomination is spoken once, followed by speech-completion cooldown.	2 runtime tests cover one announcement per scan and reset behaviour.
Interaction	Reuses obstacle detections for targets, then MediaPipe Hands finds the index fingertip. RGB/depth geometry calculates dx, dy, dz to target.	4 x 5 electrode grid becomes a 20-bit HTTP pattern. Success causes a pulse and speech.	4 runtime tests cover grid size, phase thresholds, dwell time, and invalid-depth filtering.
Audio	pyttsx3 or Windows SAPI worker with a priority queue; pygame provides alert tones and stereo panning.	Asynchronous prioritized speech; expiry avoids late messages.	1 runtime test covers banknote speech completion event and label wording.

4. Detailed operational analysis

Navigation

Detected objects are enriched with depth, viewing angle, urgency, and tracking data. Danger is nominally below 1.5 m; warning spans 1.5-3 m. A warning obstacle can be promoted to danger when it overlaps the collision corridor and is close enough. Interaction-only classes are deliberately removed from user-facing obstacle alerts but retained for object-reaching guidance. The VLM scene description is started only when there are no danger or warning obstacles, preventing conversational context from competing with safety alerts.

OCR

OCR is architected to avoid stalling the camera loop. The ControlUnit launches the reading operation in a daemon thread, while OCRReader itself uses an inference lock. Results are filtered by confidence, text height, and alphanumeric density, then ordered into readable lines. Arabic output reverses block order when it is predominantly Arabic, allowing text-to-speech to follow right-to-left reading order.

Face ID and storage

Face registration captures frames for roughly two seconds, filters weak, small, or blurry detections, averages the best compatible embeddings, normalizes the result, and persists it under a unique name. Identification compares a live embedding against all stored embeddings. This is a local-first design: no face image or embedding is sent to a remote service by this code path.

Interaction and haptics

The interaction pipeline computes a 3-D fingertip-to-target offset. It has four phases: IDLE when a hand or target is missing, GUIDANCE for general direction, EDGE when image/depth alignment is close, and SUCCESS when both image-space and depth thresholds are met. HTTP sends are asynchronous and duplicate haptic patterns are suppressed to reduce local-server traffic.

5. Test execution results

Command executed

python -m pytest -q, using the bundled Python 3.12 environment after the project virtual environment was found to reference a missing Python 3.10 installation.

Metric	Actual result
Discovered active runtime tests	23 test functions in five files
Executed tests	0 - pytest stopped during test collection
Collection errors	5
Elapsed time	0.27 seconds
Overall result	NOT PASSING in the available environment; no unit-test pass claim can be made

Collection blockers

Test area	Blocking missing module
Audio runtime	pyttsx3
Banknote runtime	cv2
Interaction runtime	pydantic_settings
Obstacle runtime	pydantic_settings
OCR runtime	cv2

Interpretation

This outcome is an environment failure, not evidence that the five features are logically failing. The test suite could not import the application modules, so no assertions were executed. The stale .venv is also a reproducibility problem: its interpreter path points to a removed Python 3.10 installation. Full test results require rebuilding the environment from requirements.txt with a supported interpreter, then rerunning pytest.

What the tests are designed to validate

- OCR language routing, GPU fallback, caching, stable-text normalization, depth prioritization, and reading order.
- Obstacle urgency promotion, invalid-depth handling, merging two detector outputs, tracker confirmation, and VLM failure shutdown.
- Interaction grid dimensions, phase boundaries, success dwell time, and invalid-depth rejection.
- Banknote one-time announcement and reset behaviour; audio speech-completion signalling.

6. Findings and implementation risks

Finding 1 - Haptic implementations are inconsistent

The active interaction path uses a safe 4 x 5 grid and a 20-bit HTTP bridge. A separate generic haptic_feedback module also starts at application startup, but it is configured for a 4 x 5 grid while its pattern_right function writes column 5 and pattern_down writes row 4. Both indices are out of range for a 4 x 5 array. The generic module defaults to stub mode, which reduces immediate runtime impact, but its directional patterns will fail if those functions are used.

Finding 2 - Banknote distance requirement is not enforced in the active mode flow

The configuration defines BANKNOTE_MAX_DIST_MM = 500 and the BanknoteDetector exposes is_note_in_range(). However, the ControlUnit's banknote probe calls detect_banknote() and the Banknote handler calls classify_denomination(); neither calls is_note_in_range(). Consequently, banknote mode can be entered and a denomination announced without the configured distance gate being applied.

Finding 3 - Test infrastructure is currently non-reproducible

The checked-in virtual environment cannot launch because it targets a missing Python 3.10 executable. The alternative bundled interpreter lacked runtime dependencies. Automated tests therefore cannot currently prove correctness until the environment is rebuilt.

Finding 4 - Test scope is incomplete for key integrations

The active pytest suite has strong logic coverage for OCR, obstacles, interaction, banknotes, and audio, but it has no active runtime tests for Face ID, the ControlUnit orchestration loop, camera acquisition, actual model loading/inference, the local haptic endpoint, or real audio playback. Several older standalone self-tests are explicitly excluded from pytest collection.

Recommended next validation steps

- Recreate .venv with a supported Python version and install requirements.txt; rerun pytest and record pass/fail counts.
- Fix or consolidate the two haptic implementations, then add tests for right/down directional patterns and endpoint payloads.
- Call is_note_in_range() before entering or processing Banknote mode if close-range scanning is a product requirement.
- Add mocked ControlUnit integration tests for priority transitions and hardware-adapter failure paths.
- Run a hardware acceptance test with OAK-D, audio output, haptic bridge, trained models, and Ollama; capture FPS, latency, recognition accuracy, and safety-alert response time.

Conclusion

The codebase implements a credible modular sensory-assistance pipeline with thoughtful asynchronous processing, safety-prioritized mode selection, and useful unit-level test design. Its current limitation is verification readiness rather than a lack of feature implementation: the environment blocks test execution, and the haptic and banknote-distance paths contain concrete implementation issues that should be resolved before claiming system-level validation.